

# Berkeley Algorithm for Clock Synchronization

---

## Problem Statement

**4.1 Implement Berkeley algorithm for clock synchronization using two physical machines. Perform operations on 2 different machines.**

---

## What is Berkeley Algorithm?

The Berkeley Algorithm is a distributed algorithm used to synchronize clocks across multiple computers (nodes) in a network. It works by having one node act as a **master** that collects time information from all other nodes (slaves), calculates an average time offset, and then instructs all nodes to adjust their clocks accordingly.

### Key Characteristics:

- **Centralized Approach:** One master node coordinates the synchronization
  - **Consensus-Based:** Uses average of all clock readings to determine the synchronized time
  - **Fault-Tolerant:** Can tolerate faulty clocks by excluding outliers
  - **Simple Implementation:** Easy to understand and implement
- 

## How Berkeley Algorithm Works

### Step-by-Step Process:

1. **Master Requests Time:** The master broadcasts a request to all slave nodes asking for their current time
2. **Slaves Respond:** Each slave responds with its current time
3. **Master Calculates Average:** The master computes the average time offset between its own time and all slave times
4. **Master Sends Correction:** The master sends the calculated correction factor to all slaves
5. **Slaves Adjust:** Each slave adjusts its clock by adding the correction factor

### Formula:

$$\text{Average Difference} = (\text{Sum of all (Client Time - Master Time)}) / (\text{Number of Clocks})$$
$$\text{Synchronized Time} = \text{Master Time} + \text{Average Difference}$$

---

## Code Explanation

### Server.py (Master Node)

```
import socket
import datetime
```

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('localhost', 8080))
server.listen(5)
print("Master Server Started...")
```

- Creates a TCP server socket on port 8080
- Listens for incoming connections from slave nodes

```
while True:
    client_socket, addr = server.accept()
    print(f"\nClient connected: {addr}")

    master_time = datetime.datetime.now()
    print(f"Master Time: {master_time}")

    data = client_socket.recv(1024).decode()
    client_time = datetime.datetime.strptime(data, "%H:%M:%S.%f")
    print(f"Client Time: {client_time}")
```

- Accepts connection from a slave
- Records the master's current time
- Receives the slave's current time

```
# Calculate average (midpoint between master and client times)
avg_diff = (client_time - master_time).total_seconds() / 2
synchronized_time = master_time + datetime.timedelta(seconds=avg_diff)
print(f"Synchronized Time: {synchronized_time}")

client_socket.send(str(synchronized_time).encode())
client_socket.close()
```

- Calculates the time difference between the two clocks
- Computes the synchronized time as the midpoint
- Sends the synchronized time back to the slave
- Closes the connection

## Client.py (Slave Node)

```
import socket
import datetime

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('localhost', 8080))

my_time = datetime.datetime.now().strftime("%H:%M:%S.%f")
print(f"My Current Time: {my_time}")
```

```
client.send(my_time.encode())

sync_time = client.recv(1024).decode()
print(f"Synchronized Time from Master: {sync_time}")

client.close()
```

- Connects to the master server on port 8080
- Sends its current time to the master
- Receives the synchronized time from the master
- Closes the connection

---

## How It Relates to the Problem Statement

### Requirement: Two Physical Machines

The code is designed to run on two different machines:

#### Machine 1 (Server/Master):

- Runs `server.py`
- Listens for connections from the client
- Acts as the time authority
- Calculates synchronized time

#### Machine 2 (Client/Slave):

- Runs `client.py`
- Connects to the master server
- Sends its local time
- Receives and displays synchronized time

### Synchronization Process

1. Both machines may have different clock times initially
2. Client connects to server and sends its current time
3. Server calculates the average between its time and client's time
4. Server sends back the synchronized time
5. Both machines can now use this synchronized time

---

## Running on Two Physical Machines

### Setup:

1. **Machine 1 (Master)** - Note its IP address (e.g., `192.168.1.100`)
2. **Machine 2 (Slave)** - Need Master's IP address

### Machine 1 - Terminal 1:

```
python3 server.py  
# Output: Master Server Started...
```

## Machine 2 - Terminal 2:

Modify `client.py` to connect to Master's IP:

```
client.connect(('192.168.1.100', 8080)) # Replace with Master's IP
```

Then run:

```
python3 client.py  
# Output will show synchronized time
```

---

## Example Output

Master (Server.py):

```
Master Server Started...  
  
Client connected: ('192.168.1.105', 54321)  
Master Time: 2026-04-28 14:30:45.123456  
Client Time: 14:30:42.987654  
Synchronized Time: 2026-04-28 14:30:44.055555
```

Slave (Client.py):

```
My Current Time: 14:30:42.987654  
Synchronized Time from Master: 2026-04-28 14:30:44.055555
```

---

## Advantages of Berkeley Algorithm

-  **Simple:** Easy to implement and understand
-  **Efficient:** Works well with small to medium number of nodes
-  **Consensus-Based:** Uses democratic approach for synchronization
-  **Fault-Tolerant:** Can ignore faulty clocks

## Limitations

- **✗ Single Point of Failure:** Master failure stops synchronization
  - **✗ Scalability:** Performance degrades with many nodes
  - **✗ Network Dependent:** Requires reliable communication channel
  - **✗ Latency Sensitive:** Network delays affect accuracy
- 

## Conclusion

The Berkeley Algorithm provides a practical solution for clock synchronization in distributed systems using a master-slave architecture. This implementation demonstrates how two machines can synchronize their clocks through network communication, making it useful for distributed applications that require temporal coordination.